# Course overview and introduction to Python

Python Programming for Business Intelligence

Sippo Rossi
March 18th 2026

# Agenda

1. Course practicalities
2. Generative AI and programming
3. Introduction to Python
4. Jupyter Notebooks
5. Git and GitHub for version control

# Learning goals for this week

**Theoretical:**

- Understand how this course works
- Understand the fundamentals of Python and Jupyter Notebooks as the development environment
- Understand the concept of version control (Git)

**Practical:**

- Be able to launch a Jupyter Notebook and run Python code
- Be able to solve basic programming tasks using Python
- Be able to save and share your code using GitHub

# Course practicalities

# About the instructors

**Sippo Rossi**

- Lecturer and examiner
- Besides teaching databases and programming, I spend most of my time researching various applications of AI
- Will answer email during working hours (sippo.rossi@hanken.fi)

**Kim Malmsten**

- Teaching assistant
- Studying computer science at University of Helsinki
- Responsible for grading assignments together with Sippo

# What this course is

This course is a thorough and rigorous introduction to using Python as a tool for data analysis and business decisions

This is course is intended for people who are genuinely interested in learning programming and working with data

This is a challenging course that rewards patience and hard work, giving you valuable practical skills

# What this course is not

This course is **not** intended as a first course in programming. **Prior experience using R or some other language is assumed**.

This is not an easy elective that you can pass with minimal effort. Either you go all in or you will not have a good time.

My other course, Introduction to Databases, has a pass rate of less than 50%, and it is easier than this course. People who passed the course rated it 4.4 / 5 though ☺

# How to succeed in this course

This course requires consistent work each week. Read the materials and complete the exercise set every week

Start working on the assignments and project as early as possible. Do not leave them to the last weekend

Always first do the programming tasks yourself and only then use AI to improve your solution

Come to the lectures and exercise sessions and ask questions. Use also the Moodle forum to ask questions

# How to become a good programmer

Programming is not difficult. It just requires the right mindset and a lot of practice

A huge part of programming (even when using AI to write the code) is related to being able to independently solve problems by trial and error

You cannot learn coding by being shown how to do things or by being given the correct solution. You must spend a lot of time practicing and trying things yourself

Being able to search for information on how to solve a specific problem and reading documentation is a big part of the process

# Generative AI and programming

# Use of AI during this course

You are allowed to use AI to help with the coding tasks in the assignments and project during this course

Use of AI in the peer review is strictly forbidden. Nobody wants to read AI slop from their classmates, and you will not learn anything by doing it with AI

Use of AI during the exam is not allowed. Being caught using AI during the exam will result in being removed from the course

# Using AI the smart way

In a basic course like this, you can practically complete all tasks easily using AI. However, you will not learn anything this way. Read more here: www.anthropic.com/research/AI-assistance-coding-skills

Always try to complete the coding tasks first by yourself, e.g., a minimum of 15-30 minutes of trying before resorting to AI

If you hit a wall, ask AI to explain the error message or what is wrong in your code if there is no error

After completing a task, use AI to ask if your own code could be improved. Note though, sometimes AI might suggest overly elaborate ways to write functionally the same piece of code

Always make sure you go through line-by-line AI generated code and that you can explain what every single command and character does

Copilot is usually really bad. No developer that I know would willingly use it. I recommend using e.g. Le Chat or Claude instead

# More reasons to code without using AI

Interviews for roles as a data or business analyst often include a technical interview where you are not allowed to use AI

The course exam mimics these interviews and there you need to be able to read and write code without access to AI

Think of this from an employer’s point of view. Why should you be hired if you are practically a slow and expensive way to ask ChatGPT?

By learning how to code, rather than just by copy pasting from AI, you can provide value. If you can only work by using AI, you are replaceable by AI

# Introduction to Python

# Python

A general-purpose programming language

Has become the most used language in data science

Syntax is clean and easy to read and write

Has an active, broad community developing everything from learning materials to new libraries

# Python vs R

Most of you should have previously used R in earlier courses.

Some differences between Python and R syntax:

- **Indexing (R starts at 1, Python at 0)**
- R uses {} to delimit code blocks. Python uses : and indentation.
- R requires () around conditions. Python doesn't.

```r
# R
x <- c(10, 20, 30)
x[1]          # 20? No — 10. R starts at 1.
x[2:3]        # 20, 30 (inclusive on both ends)
```

```python
# Python
x = [10, 20, 30]
x[0]          # 10. Python starts at 0.
x[1:3]        # [20, 30] (exclusive on the right end)
```

# Python vs R

Most of you should have previously used R in earlier courses.

Some differences between Python and R syntax:

- Indexing (R starts at 1, Python at 0)
- **R uses {} to delimit code blocks. Python uses : and indentation.**
- R requires () around conditions. Python doesn't.

```r
# R
if (x > 5) {
  print("big")
} else {
  print("small")
}
```

```python
# Python
if x > 5:
    print("big")
else:
    print("small")
```

# Python vs R

Most of you should have previously used R in earlier courses.

Some differences between Python and R syntax:

- Indexing (R starts at 1, Python at 0)
- R uses {} to delimit code blocks. Python uses : and indentation.
- **R requires () around conditions. Python doesn't.**

```r
# R
for (i in 1:5) { print(i) }
while (n > 0) { n <- n - 1 }
```

```python
# Python
for i in range(1, 6): print(i)
while n > 0: n -= 1
```

# Important concepts

Make sure you understand the following concepts very well after the first 1-2 weeks of the course:

- Variables and data types
- Expressions, statements
- Data structures: list, dictionary, tuple
- Conditional statements
- Loops: while, for
- Functions
- Objects, methods, classes

These will be explained in Jupyter Notebook file: L1_*python_basics.ipynb*

# 10-minute break

# Jupyter Notebooks

# Jupyter Notebooks

The Jupyter notebook is a browser-based graphical interface to the IPython shell (Interactive Python)

Jupyter notebooks allows the user to include formatted text, static and dynamic visualizations, mathematical equations and more in the same working space where code is written

It is widely used by researchers, practitioners and by individuals for personal projects

See the *L1_python_basics.ipynb* as an example of a Jupyter Notebook file

# Jupyter Notebook and Python demo

**Going through *L1_python_basics.ipynb***

# File formats .py vs ipynb

The standard file format for Python programs is .py (for example, *my_program.py)*

The standard file format used by Jupyter Notebooks is .ipynb (for example, *final_project.ipynb*)

Except for several special commands that can be used only in Jupyter Notebooks, the Python code in .ipynb files and .py files are fully interchangeable

# Jupyter Notebooks can be used with many tools

If you have previously worked with .ipynb files through some other tool, you can use it instead of Jupyter Notebooks, as long as you submit files in the right format. Below are some examples:

**JupyterLab**

Contains more advanced features and customizability

**Google Colab**

A cloud-based environment that requires no local installing on your computer

**Visual Studio Code (VS Code)**

Many IDEs such as VS Code also support running Jupyter Notebooks

# Git and GitHub for version control

# Version control and git

Git is the most popular version control system for developers

It is used for tracking changes to code and making backups as well as for collaborating on a shared codebase

Important terms for this course:

- **Repository**: A folder where Git tracks a project
- **Commit**: A snapshot (save) of your project at a specific time (this is the core action in Git)
- **Push:** sends the local committed changes to a remote repository (e.g. on GitHub)
- **Clone**: creates a local copy of an existing remote Git repository (e.g. from GitHub)

Why is Git useful? Let's say you work on your final project and at the end of each working session you commit the latest updates. Then on day 5 something breaks and you are unable to fix the problem. With Git you can revert to any previously saved version

# GitHub

GitHub is a cloud-based platform where you can store, share, and collaborate with others

Git is not the same as GitHub. You work locally with Git (the version control system) to track changes and then use GitHub as a tool to connect local files to a “remote” repository where you store/share code

While GitHub is primarily intended for coding, many use it also for storing backups e.g. for non-coding related projects or for sharing course materials

GitHub repositories can be **private** (seen only by you and who you give access) or **public** (seen by anyone). Never put private information into a public repository!

# Conclusion

# Conclusion

This lecture provided you with an overview of the tools that you will be working with during this course

Programming is something you learn by doing. Even if it feels intimidating now, if you write code several times a week for the next few weeks, you will start feeling comfortable

On Friday we will practice coding with Python as well as set up your first GitHub repository

# Resources

If you are not familiar with Python and feel in general unsure of your programming skills, I recommend going through **A Whirlwind Tour of Python** by Jake VanderPlas: https://github.com/jakevdp/WhirlwindTourOfPython

Depending on your familiarity with Python, you can recap topics from this website. E.g. topics under the headings *Basics*, *Functions*, and *Data Structures:* https://www.geeksforgeeks.org/python/python-programming-language-tutorial/

Downloading and installing Anaconda and launching Jupyter Notebooks. Follow the guide for Anaconda: https://www.codecademy.com/article/install-jupyter-notebook-on-mac-and-windows

To install, setup and get familiar with Git and Github, read the following: https://docs.github.com/en/desktop/overview/about-github-desktop AND https://docs.github.com/en/desktop/overview/getting-started-with-github-desktop